a. TODO 1 Memasukkan Kurung Buka ke Stack

```
s.push(char)
```

Kode tersebut digunakan untuk memasukkan karakter kurung buka (ke dalam Stack. Karakter ini disimpan sementara karena harus memiliki pasangan kurung tutup.

b. TODO 2 Memeriksa Stack Kosong

```
return False
```

Kode ini dijalankan ketika ditemukan kurung tutup) saat Stack kosong. Kondisi tersebut menunjukkan bahwa tidak ada kurung buka yang dapat dipasangkan, sehingga ekspresi dinyatakan tidak seimbang.

c. TODO 3 Menghapus Kurung Buka dari Stack

```
s.pop()
```

Kode ini digunakan untuk menghapus kurung buka yang sebelumnya dimasukkan ke dalam Stack ketika ditemukan kurung tutup. Dengan demikian, satu pasangan kurung dianggap telah terpenuhi.

d. TODO 4 Memeriksa Kondisi Akhir Stack

```
return s.is_empty()
```

Kode ini memeriksa apakah Stack kosong setelah seluruh karakter ekspresi selesai dibaca. Jika kosong, fungsi mengembalikan True, sedangkan jika masih ada kurung buka yang tersisa, fungsi mengembalikan False.

2. Hasil Program

Setelah seluruh bagian TODO dilengkapi dan program dijalankan, diperoleh output berikut:
"(3 + 5) * (2 - 1)" → Seimbang: True
"((4 + 2) * 3" → Seimbang: False
")(1 + 2)" → Seimbang: False

```python
    def __init__(self):
        self.items = []

    def push(self, data):
        self.items.append(data)

    def pop(self):
        if self.is_empty():
            return None
        return self.items.pop()

    def is_empty(self):
        return len(self.items) == 0


def is_balanced(expression):
    s = Stack()                         # Buat stack baru

    for char in expression:             # Iterasi setiap karakter
        if char == '(':                 # Jika kurung buka...
            # TODO 1: push karakter ke stack
            s.push(char)
            # pass

        elif char == ')':               # Jika kurung tutup...
            if s.is_empty():            # Jika stack kosong, tidak ada pasangannya
                # TODO 2: return False (tidak seimbang)
                return False
                # pass
            # TODO 3: pop dari stack (buang pasangan kurung buka)
            s.pop()

    # TODO 4: Setelah loop selesai, kembalikan True jika stack kosong, False jika tidak
    return s.is_empty()


# ===== MAIN =====
ekspresi1 = "(3 + 5) * (2 - 1)"
ekspresi2 = "((4 + 2) * 3"
ekspresi3 = ")(1 + 2)"

print(f'"{ekspresi1}" → Seimbang: {is_balanced(ekspresi1)}')
print(f'"{ekspresi2}" → Seimbang: {is_balanced(ekspresi2)}')
print(f'"{ekspresi3}" → Seimbang: {is_balanced(ekspresi3)}')

# Output yang diharapkan:
# "(3 + 5) * (2 - 1)" → Seimbang: True
# "((4 + 2) * 3" → Seimbang: False
# ")(1 + 2)" → Seimbang: False
```

```
"(3 + 5) * (2 - 1)" → Seimbang: True
"((4 + 2) * 3" → Seimbang: False
")(1 + 2)" → Seimbang: False
```